EECE2310 Introduction to Digital Design and Computer Architecture
Summer 2 2025 Semester

Homework 3

EECE2310 Course Section 01

Name: Hang Hang
NU ID: 002148205

Date Submitted:
7/29/2025

Q1 (16 Points)

Implement a predefined sequence counter that counts 7, 2, 5, 6 in a circular manner when an external "enable" (E) input is on. When E is off, the counter does not progress. You need to use the following two approaches to implement this counter.

a) The standard approach of designing a sequential circuit where you start with deriving the state diagram and end with the drawing of the logic diagram (with flip flops and gates). You must use the minimum number of *D* flip flops for this problem.

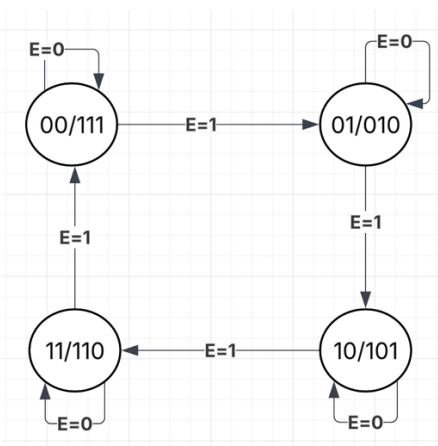
b) The approach presented at the end of the *Registers and Counters* slides where a register, an adder, and muxes are utilized. Make sure to explain in the diagram of your design the exact sizes of these components and the size of their input and output wires. You also must use the minimum number of multiplexers and minimum sizes of the register and adder needed.

(a) Standard approach

State encoding:

S0: 7 (00) --> S1: 2 (01) --> S2: 5 (10) --> S3: 6 (11) --> reset to S0

<u>State Diagram (7 = "111", 2 = "010", 5= "101", 6= "110")</u>



<u>State Transition Table</u>

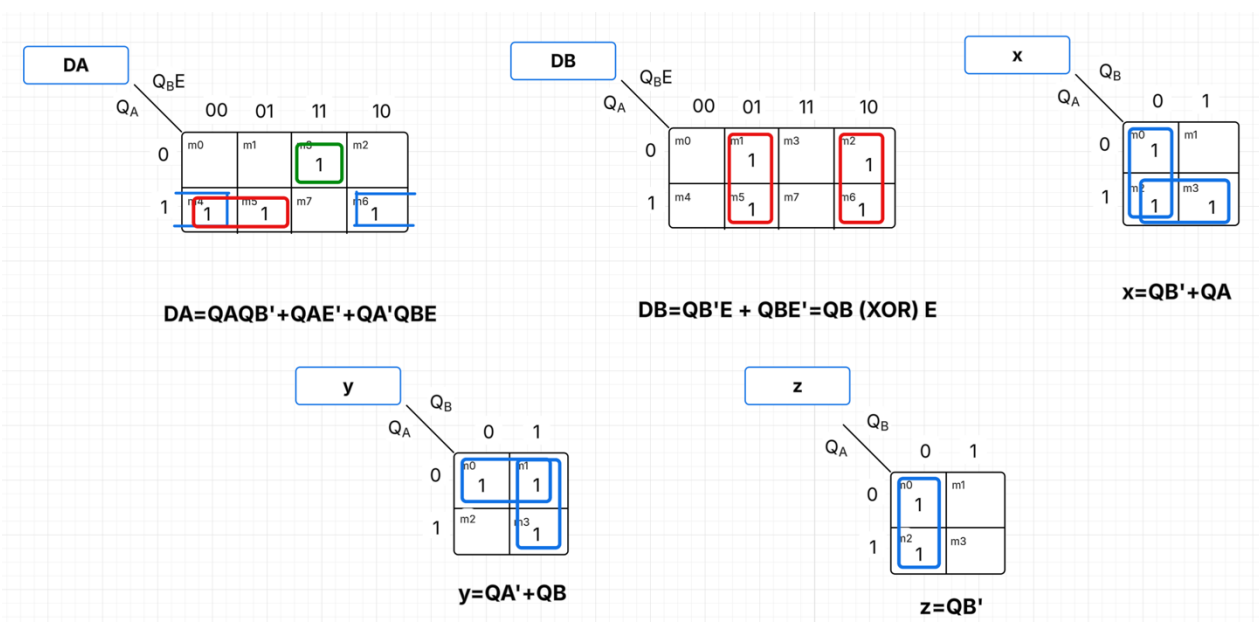
Current State		Input	Output	
QA	QB	Enable	DA	DB
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	1	0

1	0	1	1	1
1	1	0	1	1
1	1	1	0	0

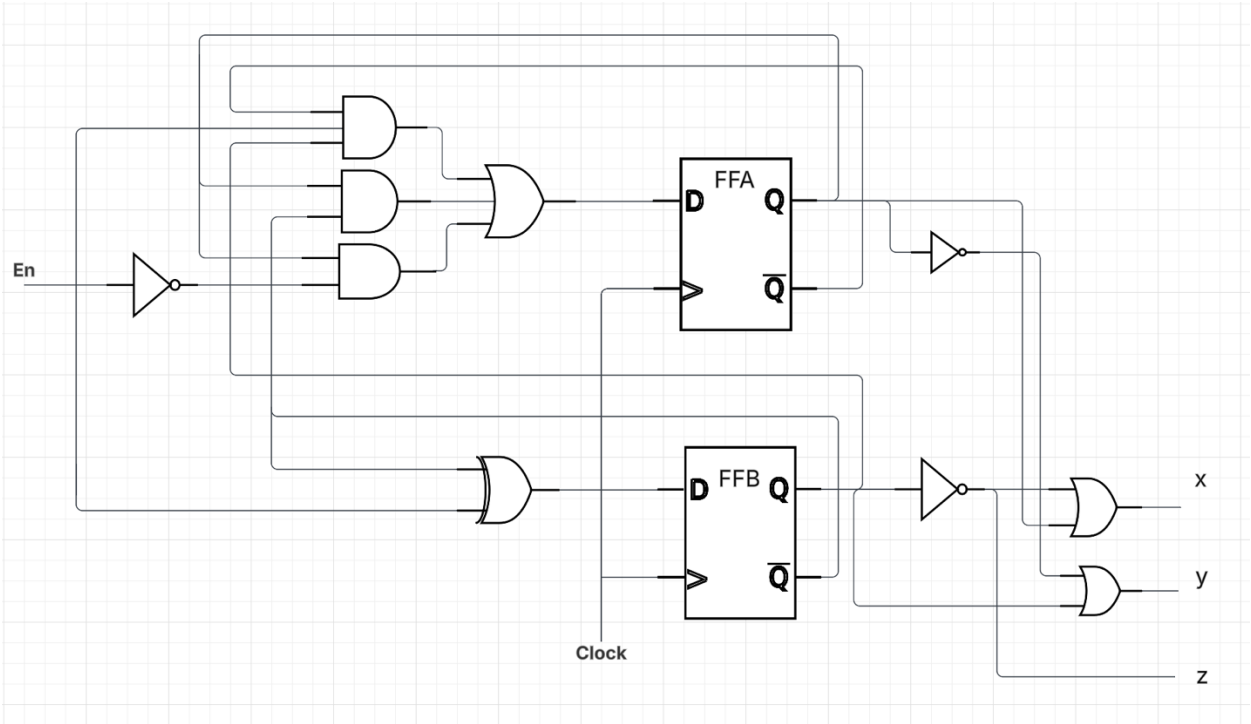
Output truth table

Current State		Outputs		
QA	QB	x	y	z
0	0	1	1	1
0	1	0	1	0
1	0	1	0	1
1	1	1	1	0

K-map



Circuit



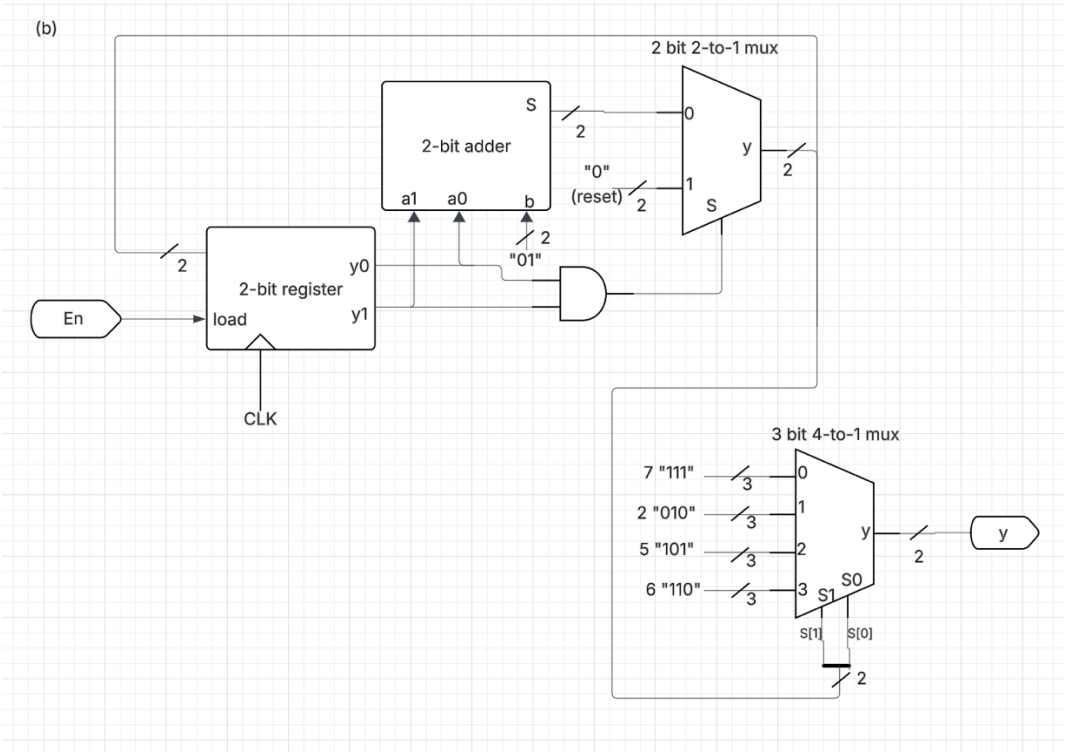
(b) State assignments are the same as part a

Register: 2 bits (as we only got 4 states)

Adder: 2 bits (add 1 each cycle)

2-to-1 Mux: 2 bits; adder output (y1 y0); hold current value vs. advance

4-to-1 Mux: 3 bits output decoder (map internal states to actual output)



- Adapted the reset logic from the lecture slide but here is a 2-bit binary version --> "01" (decimal 1).
- The 1st MUX controls internal counting (increment or reset).
- The 2nd MUX decodes the internal counter state into our predefined output #s.

Q2 (12 Points)

For each of the RISC-V instructions in the following assembly program, complete the table with the **decimal** value of its op, rd, rs1, rs2 fields. If a field is not applicable for an instruction, write N/A. Refer to the *RISC-V Reference Cards* document in the *Miscellaneous Resources* page on the course Canvas site.

Program	op	rd	rs1	rs2
`addi t0, t1, -10`	19	5	6	N/A
`lw s0, -3(t0)`	3	8	5	N/A
`L1: bne t0, s0, L2`	99	N/A	5	8
`sw s0, 84(zero)`	35	N/A	0	8

`L2: addi t0, sp, -4`	19	5	2	N/A
`beq sp, s0, L1`	99	N/A	2	8

Q3 (12 Points)

The following are the 32-bit hexadecimal values that represent the machine code of three different RISC-V instructions. What are the names of each one of these instructions (e.g., add)? Explain your answer by showing how you extract the needed fields from each instruction code.

```
a. 0x4124d333
b. 0x40305313
c. 0x0003d263
```

a. 0x4124d333
Binary = 0100 0001 0010 0100 1101 0011 0011 0011

opcode (bits [6:0]): 0110011 --> R type
rd (bits [11:7]): 00110 (x6)
funct3 (bits [14:12]): 101 (dec 5) --> srl or sra
rs1 (bits [19:15]): 01001 (x9)
rs2 (bits [24:20]): 10010 (x18)
funct7 (bits [31:25]): 0100000 (dec 32=0x20=$2*16^1+0*16^0$) --> sra
So, (a) is sra rd,rs1,rs2 = sra x6,x9,x18.

b. 0x40305313
Binary: 0100 0000 0011 0000 0101 0011 0001 0011

opcode (bits [6:0]): 0010011 --> I type
rd (bits [11:7]): 00110 (x6)
funct3 (bits [14:12]): 101 (dec 5) --> srai or srli
rs1 (bits [19:15]): 00000 (x0)
imm[11:0] (bits [31:20]): 010000000011 (dec 1027)
To see whether srai or srli, we check imm[11:5]: 0100000 (dec 32) --> srai
Imm[4:0] --> shift amount = 00011 (dec 3)
So, (b) is srai rd,rs1,imm = srai x6,x0,3.

c. 0x0003d263
Binary = 0000 0000 0000 0011 1101 0010 0110 0011

opcode (bits [6:0]): 1100011 --> B type
imm [4:1/11] (bits [11:7]): 00100
funct3 (bits [14:12]): 101 (dec 5) --> bge

rs1 (bits [19:15]): 00111 (x7)
rs2 (bits [24:20]): 00000 (x0)
imm[12/10:5] (bits [31:25]): 0000000
imm combined = 000000000100 (dec 4)
So, (c) is = bge rs1,rs2,imm = bge x7,x0,4

Q4 (15 Points)

For the following C code, assume that:

- The `s1` and `s2` registers are used to represent the integer variables `i` and `sum` respectively.
- L is an array of integers that has been already initialized with ten integers.
- The base address of L in the memory is 0x55552222.
- Each integer in this problem is stored in a word of 4 bytes.

Convert the following C code to RISC-V instructions. The purpose of the code is to calculate the sum of the ten integers stored in L. Include your code in the homework report and no need to upload a separate code file or include sample run of your code. You can only use the instructions included in the "RISC-V Computer Architecture" slides (only assembler labels and li can be used).

```
sum = 0;
for (i=0; i<10; i++)
      sum = sum + L[i];
```

```
li s1, 0                #i=0
li s2, 0                #sum=0
li t0, 10               # loop limit
li t1, 0x55552222       # base address of L

loop:
    bge s1, t0, done    # if i >= 10, loop ends
    slli t2, s1, 2      # calculate offset = i * 4
    add t2, t1, t2      # address of L[i] = base + offset
    lw t3, 0(t2)        # load L[i]
    add s2, s2, t3      # sum = sum + L[i]
    addi s1, s1, 1      # i = i + 1
    jal zero, loop      # repeat loop

done:
```

Q5 (15 Points)

Assume that the variables in the following C code are integers that have been already stored in the following registers: `t0 = c, s0 = x, s1 = y, s2 = z`

Convert the following C code to RISC-V instructions. Include your code in the homework report and no need to upload a separate code file or to include sample run of your code. You can only use the instructions included in the "RISC-V Computer Architecture" slides (only assembler labels and li can be used)

```
switch (c) {
  case 1:  x = y + z;
           break;
  case 2:  x = y - z;
           break;
  default: x++;
           break;
}
```

```
    li t1, 1                # Check Case 1 (c==1)
    beq t0, t1, case1

    li t1, 2                # Check Case 2 (c==2)
    beq t0, t1, case2

    addi s0, s0, 1          # Default case: x++
    jal zero, done
                            # case 1
    add s0, s1, s2          # x = y + z
    jal zero, done
                            # case 2
    sub s0, s1, s2          # x = y - z
    jal zero, done

done:
```

Q6 (15 Points)

Assume register `t0` has been initialized to any random 32-bit binary number. Write the RISC-V assembly code that counts the number of 1s in `t0` and stores that count in `s0`. Include your code in the homework report and no need to upload a separate code file or to include sample run of your code. You can only use the instructions included in the "RISC-V Computer Architecture" slides (only assembler labels and li can be used).

```
    li s0, 0            # Initialize count of 1-bits to 0 (stored in s0)
    li t1, 32           # Initialize loop counter to 32 (total bits to check)
loop:
    beq t1, zero, done  # Exit loop when all bits are checked
    andi t2, t0, 1      # Isolate the LSB of t0
    beq t2, zero, skip  # If the isolated bit is 0, skip incrementing
    addi s0, s0, 1      # If bit is 1, increment the count in s0
skip:
    srli t0, t0, 1      # Shift t0 right by 1 to check next bit
    addi t1, t1, -1     # Decrement loop counter
    jal zero, loop      # Continue looping
done:
```

Q7 (15 Points)

Assume s1 has been initialized to a signed integer and hence it might have a negative value.

a) The following RISC-V code increments s1 by 1 only if the current value in s1 is within a specific range. What is this range? Explain how each of the following instructions contributes towards specifying that range.

```
        ....
        slt  t1, s1, zero
        bne  t1, zero, Done
        slti t2, s1, 8
        beq  t2, zero, Done
        addi s1, s1, 1
Done:   ....
```

b) Explain why the following RISC-V code does the same task as the code in part (a) above. Support your explanation with examples.

```
        ....
        sltiu t1, s1, 8
        beq   t1, zero, Done
        addi  s1, s1, 1
Done:   ....
```

(a)

Instruction	Notes
slt t1, s1, zero	Set less than. Checks if s1 < 0. Prevents negatives.
bne t1, zero, Done	Branch !=. If negative, skip increment.
slti t2, s1, 8	Set Less Than Imm. Checks if s1 < 8. Prevents ≥8.
beq t2, zero, Done	If s1 ≥8, skip increment.
addi s1, s1, 1	Increment explicitly only if 0 ≤ s1 < 8.

(b) sltiu: performs an unsigned comparison. It treats the signed integer s1 as an unsigned number. Using sltiu achieves same behavior as (a) by treating negative numbers as large unsigned integers.

Negative signed numbers (e.g., -1, -2, ...) become large positive numbers explicitly ($\geq 2^{31}$) and they won't get incremented. Positive numbers within 0 to 7 remain as unsigned numbers <8 and hence incremented. Positive numbers >=8 remain >=8 and are excluded from incrementing.

Signed s1	Unsigned s1 interpretation	sltiu s1,8 result	Incremented?
-1	0xFFFFFFFF (huge positive)	No	No
-5	0xFFFFFFFB (huge positive)	No	No
0	0x00000000 (0)	Yes	Yes
3	0x00000003 (3)	Yes	Yes
7	0x00000007 (7)	Yes	Yes
8	0x00000008 (8)	No	No
10	0x0000000A (10)	No	No